Australian National University

Lab Two

Implementing designs onto FPGA and Essential sequential designs

Overview

This lab completes your awareness of the basic digital design steps by following your Lab One projects through to implementation.

It will then introduce you to the world of synchronous sequential logic design, where information flow is often controlled by a combination of clocks, resets and enables. These essential signals and constructs are ever-present in FPGA designs.

Note:
This lab is extensive and will require your full attention for the entire duration. **You are highly encouraged to go through this document before attending the lab.**

When adding a file (design source, constraints and simulation) to the project, make sure the option "Copy sources into project" is checked.

In this lab, you will learn to:

- Identify and define real-world inputs and outputs for the purpose of interfacing.
- Synthesise your designs and implement them on a chip.
- Create variable output rate clocks using clock dividers.
- Implement synchronous resets to return registers to known states.
- Use enable signals to control the behaviour of different design elements.

You will also become increasingly familiar with:

- Available peripherals on the development board.
- Verilog constructs such as procedural blocks and non-blocking assignments.

Australian National University

Table of Contents

Lab Overview 3
Learning Outcomes 3
Activity 1: Implementation of the Full Adder 3
Procedure 3
Step One: Open the full adder project 3
Step Two: Plan your I/O and create a constraints file 4
Step Three: Synthesise and implement your design 6
Activity 2: Overflow counters 7
Procedure 7
Step One: Create a new project in Vivado 7
Step Two: Create a new Verilog design source 7
Step Three: Describe the behaviour of a 29-bit overflow counter in Verilog 7
Step Four: Constrain your design 8
Step Five: Synthesise, implement and deploy your overflow counter 10
Activity 3: Clock division using an overflow counter 11
Procedure 12
Step One: Create a new project in Vivado 12
Step Two: Create a new Verilog design source 12
Step Three: Simulate your clock divider 12
Step Four: Modify your code to generate an approximately 1 Hz clock 13
Step Five: Constrain your design 13
Step Six: Synthesis, implement and deploy your design on the FPGA chip 13
Activity 4: Flexible clock division and heartbeat generators 14
Procedure 14
Step One: Create a new project in Vivado 14
Step Two: Create a new Verilog design source 14
Step Three: Force the counter to reset after a specific period of time 14
Step Four: Describe the behaviour of the divided clock in Verilog 15
Step Five: Introduce a heartbeat signal output 16
Step Six: Simulate your arbitrary clock divider and HB generator 16
Step Seven: Modify your design to visualise heartbeat using an LED 16
Step Eight: Simulate your design 17
Step Nine: Setting up PMOD output to read signals using an oscilloscope 17
Step Ten: design implementation on the development board and hardware test 17
Appendix 1: using a lab oscilloscope 19
Essentials of probing a signal with the oscilloscope 19
Appendix 2: using a Basys3 board for this lab 20

 © ANU 2025

Lab Overview

Learning Outcomes

By completing this lab, you will learn to:

1. Create constraints files to specify the properties of the interfaces with physical I/Os on the development board.
2. Synthesise your design and implement it on the chip.
3. Design sequential logic processes in Verilog and create basic designs such as counters.
4. Define dedicated clock signals derived from a physical oscillator on the development board.
5. Use synchronous reset and enable signals to control sequential logic processes.

To complete this lab, you will require access to Vivado and the development Board (Zybo Z7 or Basys 3). Copies of the reference manuals are provided in Canvas.

Activity 1: Implementation of the Full Adder [3 marks, approx. 15 min]

Procedure

Step One: Open the full adder project

Open the full adder project you developed during Lab 1. Check the source content and hierarchy tree of your design. The "my_adder" module should be listed as a design source and correctly identified as the "top module" of your design. **Important**: the test bench source file should only appear in the list of simulation sources. Test benches are only useful for virtual simulation of a design and are not deployable to the hardware.

Please check the "Project part" in the Project Summary window. The part number **must** agree with the chip on your development board, otherwise your implemented design will not work. The part numbers are xc7z020clg400-1 for the Zybo Z7-20 and xc7a35tcpg236-1 for the Basys3. If the Project Summary window is hidden, you can unhide it by clicking on 'Window' → 'Project Summary'.

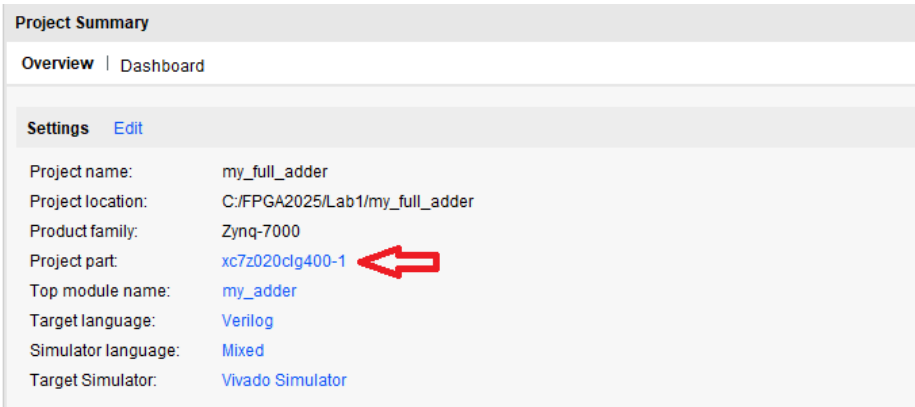


Figure 1 Project Summary window. Note the listed part number (Zybo Z7-20 here).

 © ANU 2025

Step Two: Plan your I/O and create a constraints file

To implement the design, we need to tell the synthesizer where we want our design to be placed on the chip. As we want to interact with our design, we will need to map the inputs and outputs of the design (as named in your Verilog code) to a peripheral or external pin we can physically interact with.

Hardware input/output entities are characterised by their location (an alphanumeric identifier, e.g., Y16, D18) and by a series of properties—some of which are configurable—which determine the electrical behaviour of the associated pin. A key property to be aware of is the **logic standard**, i.e., the expected voltage ranges for digital signals associated with a given pin.

The information required to match design and hardware entities is provided through a **constraints file**. This file is specific for a particular project and needs to be created by you, using a particular syntax.

The General-Purpose I/O schematic for the Zybo Z7 is shown in Figure 2 below.

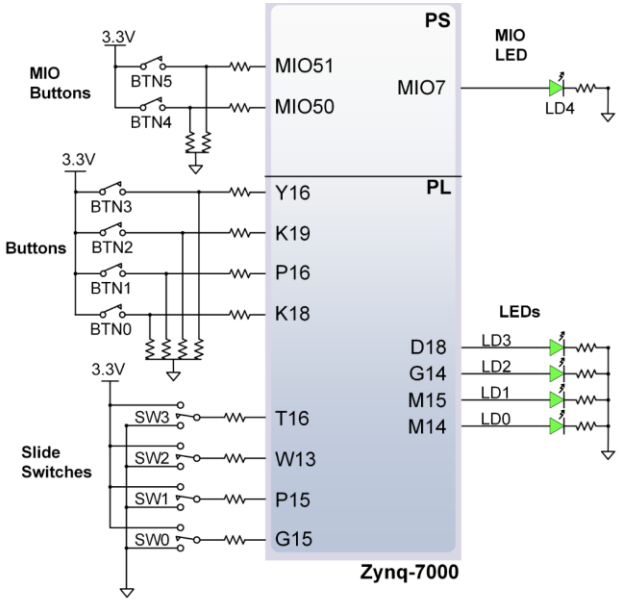


Figure 2: GPIO schematic of Zybo-Z7.

Slide switches, buttons are connected to the 3.3 V rail. The LEDs connected to pins D18, G14, M15 and M14 can be forward-biased (turned on) when a high voltage of 3.3 V is outputted by the chip at the associated pin. Note there are also some GPIO interfaces (Buttons MIO51 and MIO50, and LED MIO7) which are shown in the Figure 2 diagram but are only connected to the PS (Processing System—the microprocessor part of the Zinq SoC) and therefore cannot be used directly by the FPGA. The resistors connected in series with LEDs restrict the current flowing through the LEDs when they are forward-biased.

You can directly read the pin locations on the board. They are usually printed next to or below the hardware I/O and enclosed in brackets.

 © ANU 2025

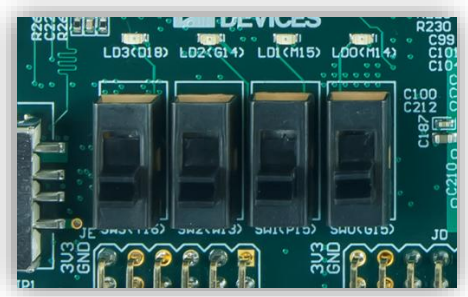


Figure 3: detail of switch/LED GPIO on Z7 board, note the LDx / SWx and alphanumeric identifiers

Now it is time for you to prepare your constraints file. **The examples in the lab manual are for Zybo Z7. If you are using Basys3, please refer to Appendix 2.**

1. Right-click the Constraints directory under the Sources panel and select 'Add Sources'.
2. Select the option to 'Add or Create Constraints' and then click Next.
3. Name your constraints file as 'full_adder_constraints_ZyboZ7' (the name is not mandated, but it is good naming practice to identify the design and the platform).
4. Once finished, the constraints file you just created should appear in the 'Constraints' directory in the Sources pane.

Design constraints inform Vivado's implementation tools of the pin location and I/O standard for all physical connections to the chip. Xilinx uses *.XDC files (XDC stands for Xilinx Design Constraints), which are written in a language called Tcl. Properties of specific interfaces can be set using the 'set_property' command.

For this activity, connect each of the inputs X, Y, and Cin to a slide switch. Connect each of the outputs Z and Cout to the LEDs.

5. To set the properties of the slide switch connected to pin T16, type in the following commands:

```
set_property PACKAGE_PIN T16 [get_ports X]
set_property IOSTANDARD LVCMOS33 [get_ports X]
```

   The `PACKAGE_PIN` property identifies the physical pin location at which we want to connect our signal. The `IOSTANDARD` property specifies the expected type of logic signal, including its logic voltage levels. LVCMOS33 stands for Low-Voltage CMOS 3.3 Volts.

6. To set the properties of the LED connected to pin D18, type in the following commands:

```
set_property PACKAGE_PIN D18 [get_ports Z]
set_property IOSTANDARD LVCMOS33 [get_ports Z]
```

7. Add the constraints for another two slide switches (one for Y, the other for Cin) and the second LED (for Cout).

Note: To comment out text in the constraints file, please use the # key. If you need to write in-line comments in the constraints file, please put a semicolon between the command and the # key.

 © ANU 2025

Step Three: Synthesise and implement your design

1. Click on 'Run Synthesis', located in the left pane of the Vivado window (Figure 4). After the design is synthesised, click on 'Run Implementation'. Finally, click on 'Generate Bitstream' and a bit-file with the file extension *.bit is generated. This file, which will have the name as your top-module, should be called 'my_adder.bit'.
2. Upon completion, Vivado will present you with a pop-up window. Select 'Open Hardware Manager' and click Next.
3. Connect your development board to your computer using the provided USB cable.
4. Power up the board. Several features of the board will light up automatically at this time. Don't be alarmed: development boards automatically load a demo design on power-up.
5. At the top of Vivado, you will notice that a green bar has appeared after opening the Hardware Manager. Click 'Open Target', and then the 'Auto Connect' icon to establish a connection with the development board.
6. Click 'Program Device'. The default design will have been replaced by the full adder. Check if the full adder behaves as intended.

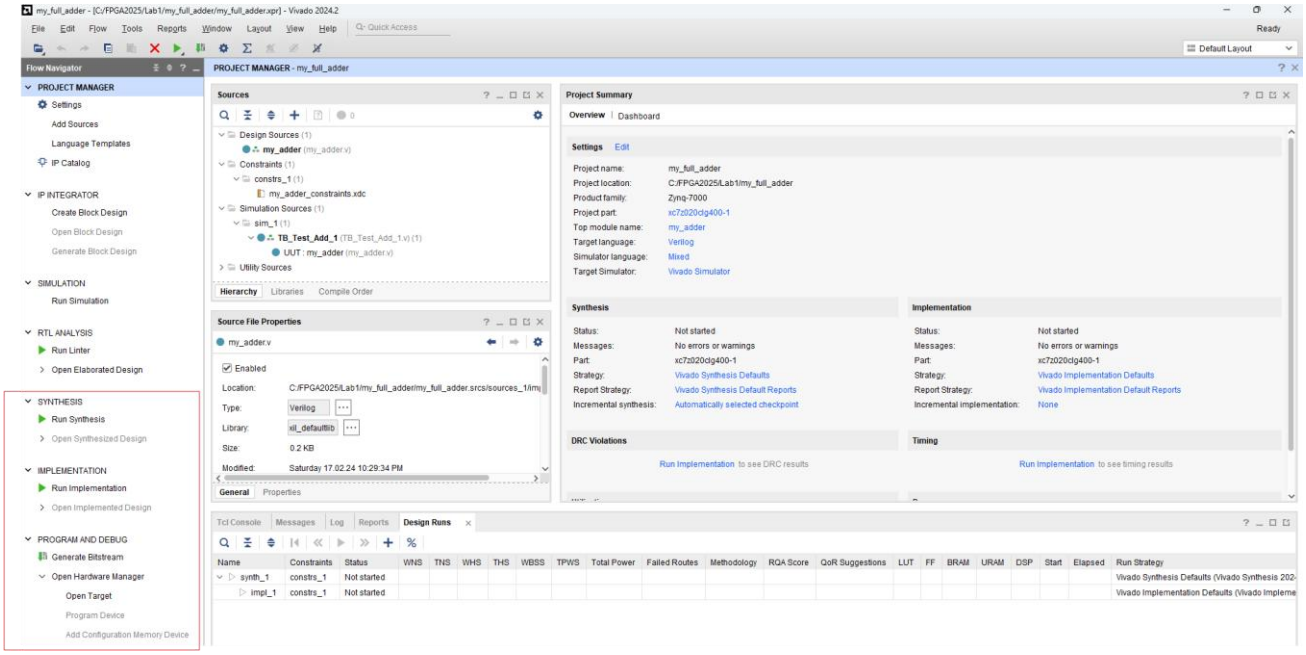


Figure 4 Vivado interface. The red rectangular box highlights where to run synthesis, implementation and programme the device.

Please demonstrate to the tutors that your design works according to the truth table for the 1-bit full adder.

 © ANU 2025

Activity 2: Overflow counters
[4 marks, approx. 30 min]

An *overflow counter* is a counter that overflows back to zero when it reaches its maximum count.

Procedure

Step One: Create a new project in Vivado

1. Create a new project called 'overflow_counter'.

Step Two: Create a new Verilog design source

1. Create a new Verilog design source 'overflowCounter'.
2. The module's inputs are `clk`, `reset` and `enable`. The module's output is `led`.

```
module overflowCounter(
        input wire clk, reset, enable,
        output wire [3:0] led
);
```

Note the syntax for the bus entity `[3:0] led`. The square brackets [3:0] defines signal 'led' as 4-bit *bus*. Individual bits of the bus may be referred to by led[n] where n is the bit number, where led[0] refers to the Least Significant Bit (LSB) and led[3] refers to the Most Significant Bit (MSB). In verilog, when using a bus as a number, the left most bit is treated as the most significant bit and the right most bit the least significant bit. The range [3:0] is specified with the highest bit number on the left of the colon to remind us as such.

We have chosen a 4-bit signal because the Zybo Z7 board has 4 available LEDs. Later, we will associate this declared bus with those LEDs.

Step Three: Describe the behaviour of a 29-bit overflow counter in Verilog

1. Declare an internal 29-bit reg called `counter`. This entity will not be part of the I/O list of signals, i.e., it will be declared in the body of the module code.

```
reg [28:0] counter;
```

Note: The use of `reg` is necessary as we will be describing the behaviour of the counter using a **procedural block**.

2. The counter should have the following behaviours:
   - 2.1 A every positive (i.e. rising) edge of the clock signal, the counter's value will increase by 1. It is achieved by a procedural block that will be edge-triggered by the clock. This is specified by the condition `posedge clk` .
   - 2.2 When `reset == 1`, the counter's value resets to 0. When `reset == 0`, the counter increments.

 © ANU 2025

2.3 The counter will increment when enabled (`enable == 1`) and stop incrementing when disabled (`enable == 0`).

3. The complete procedural block is shown here

```
always @(posedge clk) begin
      if (reset == 1'b1) begin
            counter <= 29'd0;
      end else if (enable == 1'b1) begin
            counter <= counter + 1'b1;
      end else begin
            counter <= counter; // redundant but harmless line
      end
end
```

The contents of `always @(…)` is called a *sensitivity list* and describes the conditions or events that will activate the process. This code basically says: "The behaviour described within this procedural block is triggered at every positive edge of `clk`".

This type of procedural block is called a *'clocked process'* and forms the basis of practically all sequential logic designs in FPGAs. Without clocked processes, the design cannot support dynamic states (i.e., memory, operations, events or conditions that vary over time).

IMPORTANT:

Note the use of the operator `<=` to make the assignment instead of using just an equal sign. The operator `<=` introduces a nonblocking assignment**. You should always use nonblocking assignments inside clocked processes (sequential logic)**.

Also note here the "redundant" comment in the code above. **It is good practice to explicitly define all branches when using branching statements** (if, case). We will discuss this more at a later time.

4. Assign the most significant 4 bits of `counter` to `led`. This line should be **outside** the `always` block.

```
assign led = counter[28:25];
```

With a clock at 125MHz, counter LEDs would blink too quickly for the human eye. Setting only the most significant bits of the counter as output slows down the blinking significantly.

5. Review the elaborated RTL schematic.

Step Four: Constrain your design

1. Create a constraints file called 'overflowCounter_constraints_ZyboZ7'. There are two typical ways of compiling constraints files
   a. *Obtaining and modifying a template constraints file.* A general constraints file for Zybo Z7 is available on GitHub (a copy has been uploaded to Canvas also), see [digilent-xdc/Zybo-Z7-Master.xdc at master · Digilent/digilent-xdc · GitHub]

      You can copy and paste the lines you need from the template into your constraint file. You will need to modify the names of the I/O ports so that they agree with your design source file.

      Xilinx-supplied Constraints templates use a particular syntax, where multiple

 © ANU 2025

properties are set in a single line of code. This uses the flag `-dict` followed by any required `<property><value>` pairs. For instance,

```
set_property -dict { PACKAGE_PIN D18    IOSTANDARD LVCMOS33 } [get_ports {led[3]}]
```

b. *Entering required constraints manually*. This is a traditional but equivalent way of writing the constraints one line at the time is shown below

```
set_property PACKAGE_PIN D18 [get_ports {led[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

Note: Following "b." at first is a good way for learners to appreciate each constraint. However, once you have understood how constraints work, you are encouraged to use method "a." as it can help you save a considerable amount of time in the lab.

Syntax Note: to identify a single bit in a bus for constraints file purposes, curly brackets need to be used, e.g, `{led[3]}`. The curly brackets are required because Tcl interprets anything in square brackets as a function, which means that passing an argument 'led[3]' into the `get_ports` function would cause Tcl to look for a function called '3' and ignore the square brackets as literal characters. Tcl will treat anything within curly brackets as literal characters, which allows us to achieve correct identification of our entity.

2. Add the necessary constraints for the 4-bit output led.
3. We are now going to recruit a clock for the input `clk`. We will be using the 125MHz oscillator attached to pin K17 (see bottom of page 5 in Reference Manual). The clock has a period of 8 nanoseconds.

```
set_property -dict { PACKAGE_PIN K17   IOSTANDARD LVCMOS33 } [get_ports {clk}];
create_clock -add -period 8.00 [get_ports {clk}];
```

Note that the first line is equivalent to

```
set_property PACKAGE_PIN K17 [get_ports {clk}]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

The command create_clock informs the implementation tools that the signal entering the FPGA on pin K17 is a clock and must be considered separately to the other general purpose I/O (e.g. LEDs, switches). **The creation of a dedicated clock is crucial because they are routed separately throughout the FPGA along dedicated 'clock lines', the purposes of which are to minimise latency. Importantly, <u>the period specified must match the period of the oscillator</u> (clock) being input on that port exactly. The create_clock command is not used to create a derived (non-native) clock!**

Note: The oscillator on the Basys3 Board has a frequency of 100MHz. See Appendix 2 for clock generation for Basys3.

4. Add the necessary constraints so that the *reset* signal is associated with the rightmost push button (K18).
5. Add the necessary constraints so that the *enable* signal is associated with the rightmost slide switch (G15).

 © ANU 2025

Step Five: Synthesise, implement and deploy your overflow counter

1. After the device is programmed, flick the enable switch. You should see the LEDs turning on and off at different rates. The LEDs on the left should flicker more slowly than those on the right.
2. Test the function of reset by pressing the rightmost push button.
3. Try resetting the counter while it is disabled. Does it reset the way you expect?

   Reflect on the relative priority of enable and reset signals from a system operation perspective, and on how different coding choices may impact this.

Please demonstrate to the tutors that

- The LEDs are blinking in the correct pattern.
- The reset and enable function as intended. Specifically, you should be able to reset regardless of if the system is enabled or disabled.

 © ANU 2025

Activity 3: Clock division using an overflow counter [5 marks, approx. 30 min]

In many cases, it may be necessary to generate clock signals slower than the native clock frequency. A very efficient way to do this is to perform clock division using an overflow counter.

The simplest way to generate a clock divided by a factor of 2^n is to output the (n-1)-th bit of an overflow counter. 'n-1' arises from bus indexing starting from 0. The table below should give a clear visual representation of this (look at the alternating dark/light cells down each column).

Consider a 4-bit counter being incremented at a rate of 125 MHz (period of 8 nanoseconds), which is the oscillator frequency for Zybo Z7:

Time (ns)	counter[3]	counter[2]	counter[1]	counter[0]
0	0	0	0	0
10	0	0	0	1
20	0	0	1	0
30	0	0	1	1
40	0	1	0	0
50	0	1	0	1
60	0	1	1	0
70	0	1	1	1
80	1	0	0	0
90	1	0	0	1
100	1	0	1	0
110	1	0	1	1
120	1	1	0	0
130	1	1	0	1
140	1	1	1	0
150	1	1	1	1
	7.8125 MHz	15.625 MHz	31.25 MHz	62.5 MHz

The least-significant bit (bit 0) of the 4-bit counter is a periodic signal with a frequency of 125 MHz / 2^1 = 62.5 MHz, whereas the most-significant (bit 3) bit has a frequency of 125 MHz / 2^4 = 7.8125 MHz.

So, if we want to derive a 1 kHz clock from the 125 MHz master clock, how big must the overflow counter be? Use the equation: $f_{div} = f_{clk} / 2^n$ to find out!

Notice that division by powers of two cannot yield a divided clock running at exactly 1 kHz. The closest that can be achieved is 953.67 Hz, which can be generated by observing the most-significant bit of a 17-bit counter incrementing at 125 MHz. A closer to exact clock could be

 © ANU 2025

achieved if we did not simply divide by 2^n (we will do something like this in Activity 4).

Note: If you are using Basys3, the oscillator frequency is 100MHz. While the considerations above still apply, the actual numbers may vary.

Procedure

Step One: Create a new project in Vivado

1. Create a new project 'overflow_clock_divider'.

Step Two: Create a new Verilog design source

1. Create a new Verilog design source 'overflowClockDivider'.
2. Define the module's inputs to be 'clk', 'reset' and 'enable'. Declare its output to be 'dividedClk'.
3. Inside the module, declare an internal 17-bit reg 'counter' and describe its behaviour with both the reset and enable signals using a procedural block.

```
`timescale 1ns / 1ps

module overflowClockDivider (
      input wire clk,
      input wire reset,
      input wire enable,
      output wire dividedClk
);

      reg [16:0] counter;

      always @(posedge clk) begin
            if (reset == 1'b1) begin
                  counter <= 17'd0;
            end else if (enable == 1'b1) begin
                  counter <= counter + 1'b1;
            end
      end

endmodule
```

4. Add one line of code which continuously assigns the most-significant bit of the 17-bit counter to output `dividedClk`.

Step Three: Simulate your clock divider

1. Import the testbench file called 'TEST_overflowClockDivider' from the supplied resources. **Make sure the option "Copy sources into project" is checked.**
2. Inspect the testbench file and make sure you can identify the following elements:
   a. signals `clk`, `reset` and `enable` of type 'reg', which are used to provide the test inputs
   b. a wire to read the output divided clock, called `dClk`

 © ANU 2025

c. an instance of your overflowClockDivider module, named UUT (Unit Under Test), duly connected to the relevant testbench signals

d. a behavioural block creating the master 125 MHz clock, using the keyword `forever`. The clock clk is negated every 4ns, i.e., forever **#4** clk=~clk. This results in a clock period of 8ns.

e. a sequence of assignments determining the sequence of test inputs. These are defined inside an `initial` block.

Note that if you are using Basys3, you should negate the clock clk every 5ns to create a 100MHz clock, i.e., 'forever **#5** clk=~clk'.

3. Run the behavioural simulation. When the simulation window appears, press the *'zoom-to-fit'* button at the top of the simulation window.

4. The default simulation time is probably not long enough to see if the clock divider module is behaving as designed. You can run the simulation for longer by typing in a desired simulation time and then pressing the play button with a τ symbol next to it in the menu at the top of the Vivado window.

Figure 5 Simulation toolbar showing how to extend the simulation time.

Simulate the clock divider module for an additional **10 ms** and then zoom out. Measure the period of `dClk` using two cursors (using shift + left click to generate additional cursors) and positioning them at the beginning and the end of a period of `dClk`. The measured period should be 1.048576 ms, corresponding to a frequency of approximately 953.67 Hz.

Please show the simulation results to the tutor before proceeding.

Step Four: Modify your code to generate an approximately 1 Hz clock

Modify 'overflowClockDivider.v' to generate a clock at a frequency close to 1 Hz. You will need to change the size of the counter.

Important: <u>DO NOT</u> simulate this. If you do, the simulation will take a *very* long time to complete due to the large number of timesteps (1 second is 10^9 ns!).

Step Five: Constrain your design

Create a design constraints file called 'overflow ClockDivider_constraints_ZyboZ7'.
You can save some time by copying the constraints you created for Activity 2.
You **only need to constrain one LED** as the output.

Step Six: Synthesis, implement and deploy your design on the FPGA chip

Check that the LED flashes at approximately 1 Hz. Also, please test the functions of reset and enable.

Please show the tutor that

- The LED blinks at approximately 1 Hz.
- Reset and enable function as intended.

 © ANU 2025

Activity 4: Flexible clock division and heartbeat generators
[8 marks, approx. 1 hr]

We now try to overcome the limitation of 2^n clock division. We look for an alternative design approach for a clock divider which allows for the creation of an arbitrary frequency. This involves resetting a counter every time it reaches a specific value instead of relying on its periodic overflow behaviour.

For example, if we want to derive a 1 kHz clock, then we need to generate a signal that flips between 1 and 0 every 0.5 milliseconds. If the master clock is 125 MHz, one way to do this is to increment a counter by one at 125 MHz and manually reset it to zero every time it reaches 62499 (i.e., after 62500 clock cycles). The counter will therefore reset every 0.5 milliseconds, and every time it does we can invert the current value of the clock.

Unlike the clock, which usually has a 50% duty cycle (half the time it is 1, half the time it is 0), a **heartbeat** is a signal that delivers a pulse of one clock cycle duration at regular intervals. Heartbeat generators are quite useful as **event sequencers**, i.e., when used to trigger the start of some other circuit event with precise timing.

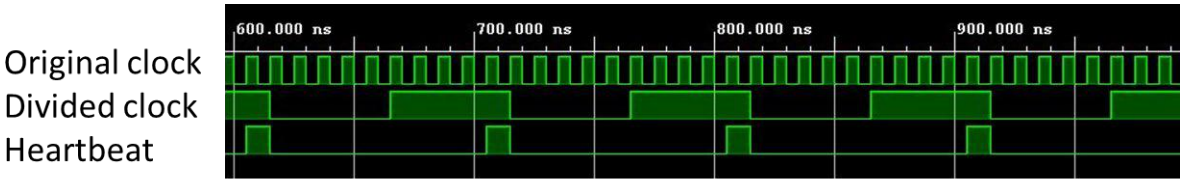


Figure 5 Example original clock, divided clock and heartbeat.

Procedure

If you are using Basys3, you can follow the same procedure but the numbers are likely to be different. Basys3 has a 100MHz oscillator.

Step One: Create a new project in Vivado

1. Create a new project 'arbitrary_clock_divider'.

Step Two: Create a new Verilog design source

1. Create a new Verilog design source 'clockDividerHB'.
2. Define the module's inputs to be `clk`, `enable` and `reset`, and its outputs to be `dividedClk` and `beat`.
3. Set the types of `clk`, `reset`, `enable`, `beat` to be 'wire', and `dividedClk` to be 'reg'.

Step Three: Force the counter to reset after a specific period of time

1. Inside the module, create a 32-bit reg entity named "counter".

```
reg [31:0] counter;
```

Our desired behaviour is to increment the counter for a specific number of clock cycles

 © ANU 2025

before resetting it back to zero. In general, we can refer to the number of clock cycles we need to count as 'threshold'. The behaviour of our periodically resetting counter can thus be described logically by the statement: "If the counter is greater than or equal to the threshold minus one, set its value to 0".

2. To declare `THRESHOLD`, we are going to use a `parameter`, which can be used to declare constants. Parameters are useful to support a powerful coding technique called **parameter substitution**.

   To generate a 1 Hz clock, at a master clock rate of 125 MHz we will need to count for 62,500,000 clock cycles each time we invert the clock. The final module declaration should look like the below

```
module clockDividerHB #(parameter integer THRESHOLD = 62_500_000)(
        input wire clk, reset, enable,
        output reg dividedClk,
        output wire beat);
```

   The reason for writing `THRESHOLD` in all caps is a style choice, which helps to quickly identify parameters in the code.

   Note the declaration of a parameter did not involve the usual notation (i.e., `<size>'<radix><value>`) to specify the numeric value. The reason for this is that `THRESHOLD` has been declared as an *integer* (a type that assists with coding but *is not implemented* in programmable logic), meaning it will be interpreted as a decimal integer by default.

   With `THRESHOLD` declared, we can describe the behaviour of the counter.

```
always @(posedge clk) begin
      if (reset==1'b1 || counter >= THRESHOLD-1) begin
            counter <= 32'd0;
      end else if (enable == 1'b1) begin
            counter <= counter + 1'b1;
      end
end
```

Step Four: Describe the behaviour of the divided clock in Verilog

1. Create a new clocked process with two conditions: one for the input signal `reset`, which should set `dividedClk` to zero if `reset` is true; and the other to invert the current value of `dividedClk` every time the counter reaches `THRESHOLD-1`.

```
1    always @(posedge clk) begin
2          if (reset == 1'b1) begin
3                dividedClk <= 1'b0;
4          end else if (counter >= THRESHOLD-1) begin
5                dividedClk <= ~dividedClk;
6          end
7    end
```

 © ANU 2025

This clocked process will execute concurrently with the one describing the behaviour of the counter, which means that both processes will respond to all logic conditions at the same time.

Note that there is no explicit `else` statement included in this procedural block. In sequential logic this is sometimes acceptable, while it is never acceptable in a combinational logic block due to inferred latch.

Step Five: Introduce a heartbeat signal output

Heartbeat generation can be obtained by introducing a simple combinatorial function which yields a TRUE output for 1 clock cycle and only once per every divided clock's period. There are countless possibilities to achieve this. Here is one:

```
assign beat=(counter==THRESHOLD-1)&(dividedClk);
```

Step Six: Simulate your arbitrary clock divider and HB generator

Please use the provided 'TEST_clockDividerHB.v' to simulate your clock divider. Pay attention to the module instantiation.

```
clockDividerHB #(.THRESHOLD(62_500)) UUT
        (.clk(clk), .reset(reset), .enable(enable), .dividedClk(dclk), .beat(beat))
```

In the instantiation, `THRESHOLD` has been overridden from its default value of 62,500,000 to 62,500. This means that your module will generate a divided clock frequency of 1 kHz. Review the simulation result and use two cursors to measure the period of `dividedClk`. **Do not simulate the 1 Hz case.**

Please show the simulation results to the tutor before proceeding.

Step Seven: Modify your design to visualise heartbeat using an LED

The heartbeat signal is too short to be picked up by the human eye if it is connected to an LED. Therefore, we need to increase the duty cycle of the heartbeat signal. Suppose we wish the duty cycle to be 20%, we need to change the logic so that the `beat` pulse is "stretched", i.e., kept HIGH for a longer period of time. To do so, declare another parameter called `ON_TIME`.

Create a new design source file called 'clockDividerHB2'. The module declaration should look like

```
module clockDividerHB #(parameter integer THRESHOLD = 62_500_000,
                        parameter integer ON_TIME = 25_000_000)(
        input wire clk, reset, enable,
        output reg dividedClk,
        output wire beat);
```

Note: The use of parameters is not strictly required. Parameters are used when they can make coding easier. In this case, having declared parameter `ON_TIME` would allow you to change the HIGH duration by editing a single number in code.

The Verilog code for the rest of module clockDivederHB will be the same as before, except for the expression for `beat`, which becomes:

```
assign beat=(counter>=THRESHOLD-ON_TIME)&(counter<=THRESHOLD-1)&(dividedClk);
```

 © ANU 2025

Note: It is not strictly correct to call the new signal a "heartbeat". It is most common for heartbeat signals to remain internal to a logic design, i.e., they typically serve a trigger function for other circuit activities and are not output to be visualised. We are creating a specialised (pseudo) 'beat' for the purposes of this exercise.

Step Eight: Simulate your design

Please use TEST_clockDividerHB2.v provided in the Canvas resources folder and perform a simulation, confirming that your design operates as intended.

Step Nine: Setting up PMOD output to read signals using an oscilloscope

So far, we have checked our designs simply by visual inspection of blinking LEDs. In this final task, we will keep the LEDs, but we will also use an oscilloscope to read the output of our digital logic. This will allow us to verify our work more precisely – an important skill for engineers!
You may have used an oscilloscope previously in ENGN2218. Some quick refresher information is provided in Appendix 1 just in case.

An oscilloscope lead needs to be hooked onto a signal-carrying pin output in order to perform measurements. Therefore, we will need to mirror of our signals of interest (`dividedClk, beat`) to an appropriate output location. We will use one of the PMOD ports for this purpose, specifically port JE. Please have a look at the reference manual, section 16, for more information. The numbering of pins of PMOD ports on the Zybo Z7 is shown in Figure 6.

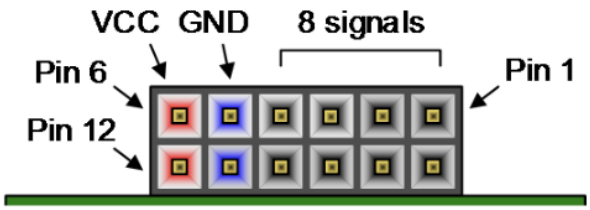


Figure 6: how PMOD pins are numbered on the Zybo Z7.

We will seek to output signal `dividedClk` from Pin #1 of PMOD port JE, and signal `beat` from Pin #7. We will plug small jumper leads into the relevant pins and connect the oscilloscope to our jumper leads.

Create a constraints file. This will need to cater for the blinking LEDs, as well as the additional mirrored outputs at PMOD port JE. You will need to consult Table 16.1 in the reference manual to figure out the correct identifier for the relevant pins. The necessary constraints specifications include the FPGA pin identifier and the logic voltage (we will use the typical Z7 standard, i.e., 3.3V).

```
set_property -dict {PACKAGE_PIN *LOCATION*   IOSTANDARD LVCMOS33} [get_ports {*entity_name*}];
```

Note that you will need to specify two of these: one for each of the output pins you are recruiting.

Step Ten: design implementation on the development board and hardware test

1. Run Synthesis, Run Implementation, Generate Bitstream, then program the device.
2. Once programmed, check visually that the LED corresponding to the 1Hz clock flashes at 1 Hz, while the LED that corresponds to the heartbeat is only ON for a very short time just before the 1Hz clock LED switches from ON to OFF.

 © ANU 2025

3. Connect jumper leads to pins #1 and #7 of PMOD port JE. Connect one further jumper lead to a ground (GND) pin (pin #5 or #11). Please **do not connect anything in the Vcc pins** (i.e., the leftmost pins #6 and #12, see Figure 5). If Vcc pins are accidentally shorted to GND, a high current will result, which could seriously damage the board.
4. Connect each oscilloscope lead to one of the signal outputs and to ground (see Figures below for additional guidance).

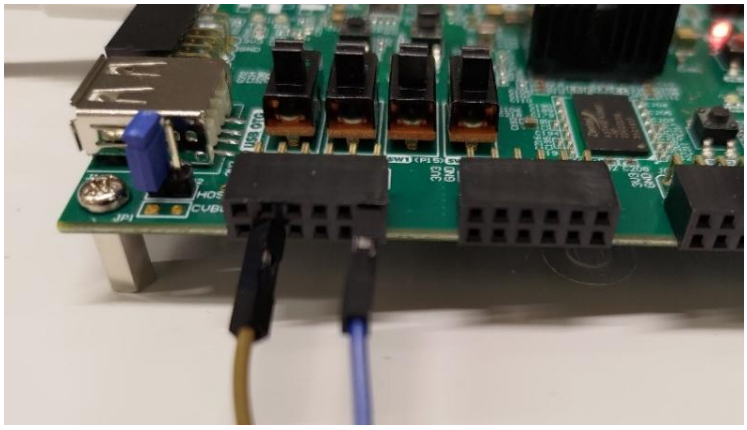


Figure 7: Connecting jumper leads to PMOD pins. In this picture, the blue jumper is connected to pin 1 and the brown jumper is connected to GND.

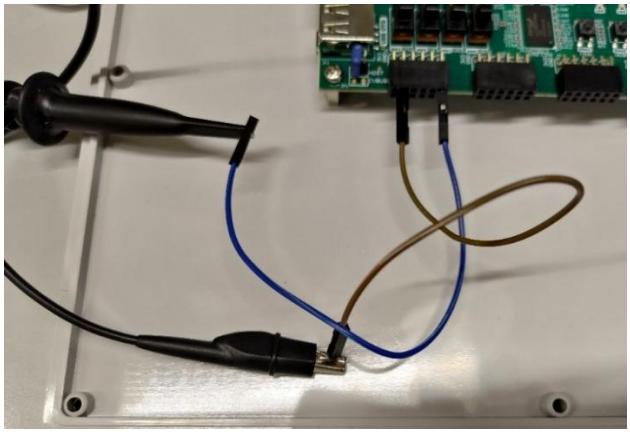


Figure 8: Connecting the blue jumper lead (signal) and the brown jumper lead (ground) to the oscilloscope probe. **Please note:** the crocodile clip (bottom in figure) must be connected to the ground lead.

5. Manipulate the timescale and amplitude settings on the oscilloscope device so as to visualise the switching signals. Use the measurement tools to verify that the frequency and duty cycle of the two signals match your expected outputs.

Please show the tutors that there are two LEDs blinking, both at 1Hz. One acts like a normal clock (i.e. with 50% duty cycle), while the other has a smaller duty cycle. Demonstrate the measurements on the oscilloscope screen and confirm that the values read are correct and coherent with expectations.

Your lab notebook should be used for task development notes and any other learning annotations that that work for you. The tutor will check that you have at least:

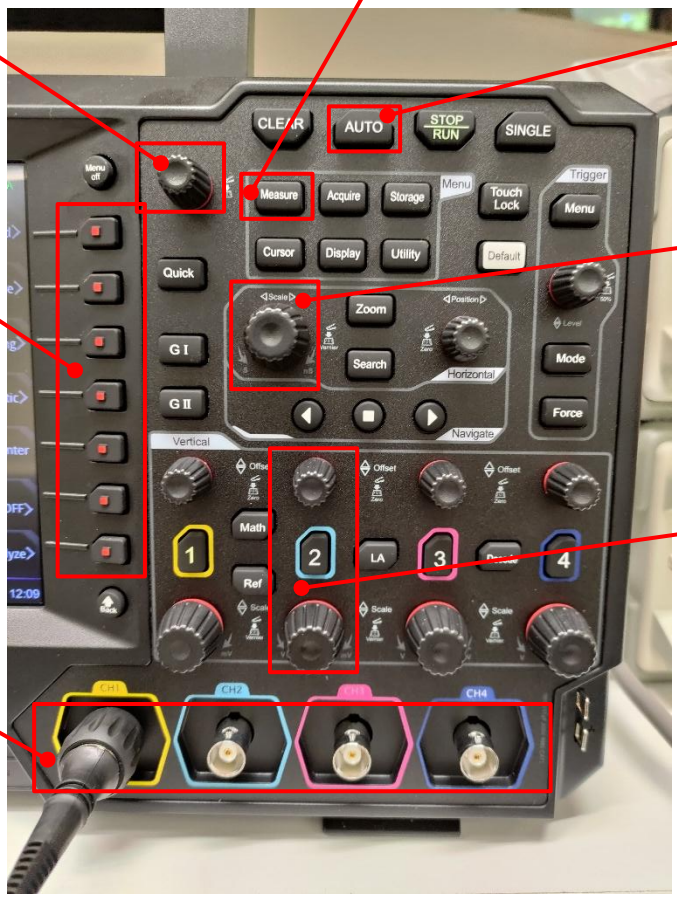
- drawn an approximate schematic (don't need to draw every single flip-flop) of the logic system you have designed for this activity, including reset and enable actions.
- some simple calculations to explain why ON_TIME = 25_000_000 results in 20% duty cycle.

End of Lab Two (there are some appendices below)

 © ANU 2025

Appendix 1: using a lab oscilloscope

Hopefully you have not forgotten how to use an oscilloscope from your early electronics years. Here is a quick refresher of the interface with basic controls. The instruction manual for the lab oscilloscopes is provided in Canvas.



Essentials of probing a signal with the oscilloscope

1. Ensure that your probe lead is connected to one of the available BNC channel connectors.
2. Connect the probe to the desired signal pin (grab hook), and to ground (crocodile clip)
3. Turn on the input for the channel you wish to use, then check the system display
4. If you are unhappy with the way in which the signal is shown, adjust vertically and horizontally as required. You can try the AUTO function too – though beware it does not always work well.
5. If your signal is not being captured, you may need to review your trigger settings. If you have not used an oscilloscope before, you can ask the tutor for help with this.
6. Once you can see the signal, you can perform measurements by pressing the "Measure" button and selecting among the many options for signal analysis shown on the screen. The measurement results will be shown on the screen, below the signals window.
7. It is possible to capture signals and screenshots using a thumb drive connected to the USB port. **Important:** DO NOT connect anything other than memory storage to the USB port.

 © ANU 2025

Appendix 2: using a Basys3 board for this lab

Most of the activities in this manual can be tried on a Basys3 board as well. Here are some of the key aspects to keep in mind when translating across the two devices:

- PART NUMBERS:
  When setting up a project, you are asked to select the FPGA part number. This informs the toolchain on how resources can be recruited within the chip to achieve the desired design. If you select the wrong part number, your design will not run. We recommend creating two separate projects: one to run on the Zybo Z7 (in the lab) and one to run on the Basys3 (at home).

- CLOCK:
  the Basys3 board runs on a 100MHz clock, while the Zybo Z7 runs on 125 MHz. This means that the clock definition in the constraints file will be different between the two. In the Basys3, the clock enters the FPGA from pin W5, and uses 3.3V CMOS logic.

  Since the clock frequency of the two boards is different, the required counts for the clock division tasks will change, depending on the faster/slower underlying clock.

- CONSTRAINTS
  the pinouts of the Basys-3 and Z7 boards are different. This means that locations used in the constraints file will need to be re-mapped so that they are correct for the specific type of board in use. The locations can be identified by looking at the Reference Manual for your board. For I/O locations, the identifiers are also usually printed on the board itself, as shown below (e.g., on the Basys 3 board, the left pushbutton BTNL is identified as W19, LED LD6 is identified as V14, etc.).

  To assist you with coding constraints files more speedily, a *constraints template file* is available for the Basys-3 board as well. A copy should be in Canvas, but here is a link to the official Xilinx repository: digilent-xdc/Basys-3-Master.xdc at master · Digilent/digilent-xdc · GitHub. Please inspect this file. You can copy/paste lines out of it as required for your design. Please ensure that you edit each line to correctly name-match your design entities.

 © ANU 2025